

DSA STUDY BLUEPRINT SERIES

41. Rat in Maze Backtracking

Finding All Path Solutions in Grid Maze

Section 06 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Problem:** Navigate from top-left (0,0) to bottom-right (N-1,N-1).
- Avoid revisiting cells by marking them as visited and unmarking them during backtrack steps.

Memory & Execution Blueprint

Paths directions: Down (D) • Left (L) • Right (R) • Up (U)

C++ Implementation Reference

Standard code layout and syntax

```
void solveMaze(int r, int c, vector<>& maze, string path, vector<string>& ans) {  
    if (r == n-1 && c == n-1) { ans.push_back(path); return; }  
    maze[r][c] = 0; // Mark visited  
    if (isSafe(r+1, c)) solveMaze(r+1, c, maze, path + 'D', ans);  
    if (isSafe(r, c-1)) solveMaze(r, c-1, maze, path + 'L', ans);  
    maze[r][c] = 1; // Backtrack  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(3^{N^2})$
Space Complexity	$O(N^2)$

Key Practices & Gotchas

- **Important:** Ensure checking boundaries before accessing maze coordinates.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace backtracking cell navigations in a grid puzzle:

Cell visited	Direction Checked	Move Validation	Dead-end Backtrack step
(0, 0)	Down (D) → (1, 0)	Valid (Open path)	Proceed to cell (1, 0)
(1, 0)	Right (R) → (1, 1)	Invalid (Blocked wall)	Revert to cell (1, 0) and try Next check

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Pruning Searches:** Check safety criteria (e.g. diagonal checks for N-Queens) to skip bad board configurations immediately.
- **Visited state maps:** Mark board grid tiles as visited inline to avoid loops, resetting them when returning.
- **Related LeetCode Problems:** #51 N-Queens, #37 Sudoku Solver, #79 Word Search.